Conversational Gen AI Application with Prompt Engineering

Assignment 1: Make your AI Chatbot session aware/conversational

The **my_first_genai_chatbot.py** streamlit app that you created answers the question asked, but it is not conversational as it does not preserve any state or memory in the application.

So, if you ask the question:

“Tell me something about TCS”

And then ask the question

“How many employees does it have?”

LLM will not be able to answer the question as the context (TCS) is not passed to it

To make it conversational, we need to

a) Add a state/memory in the application where the chat history is saved, and
b) Pass the chat history to the LLM as a context along with the prompt, so that LLM can answer with context.

Let’s do that with the help of code generation capabilities of copilot/Claude/Gemini

Claude is rated very high for code generation but you may use any other tool.

To do it

1) Copy paste your existing **my_first_genai_chatbot.py** code into claude or gemin/copilot
2) Give the following prompt (or similar):
   You are an expert python and Gen AI trainer. Modify the above code with minimal line and simplest possible code such that the AI chatbot becomes conversational (it keeps session state). The previous messages and responses should be shown in the UI just like chatgpt does.
3) Create a new file in your VS code project, named: **conversational_ai_chatbot.py**, and paste the generated code into it
4) Run the file with streamlit run command on terminal, from the project root folder:
   a) Make sure your virtual environment is active. Issue following command:
   venv\Scripts\activate
   b) Run the streamlit command:
   streamlit run <path to file>
5) Now when you ask:
   “Tell me something about TCS”
   Followed by:
   “How many employees does it have?”
   It will correctly understand you are asking about TCS from the conversation history and print the number of employees of TCS

Assignment 2: Create a conversational application using PE

You will build **a full fledged conversational application** purely by crafting and refining prompts, focusing on structured output, error handling (missing data), and domain specificity.

Each assignment illustrates how to request information from users and handle missing information before generating the final artifact.

Tasks:

Build interactive chatbot for the assignment **assigned to you**

1) You may use the promptperfect or any other chatbot to help you generate the prompt (https://promptperfect.jina.ai/)

2) Your chatbot should be context aware (have memory/history) for the session

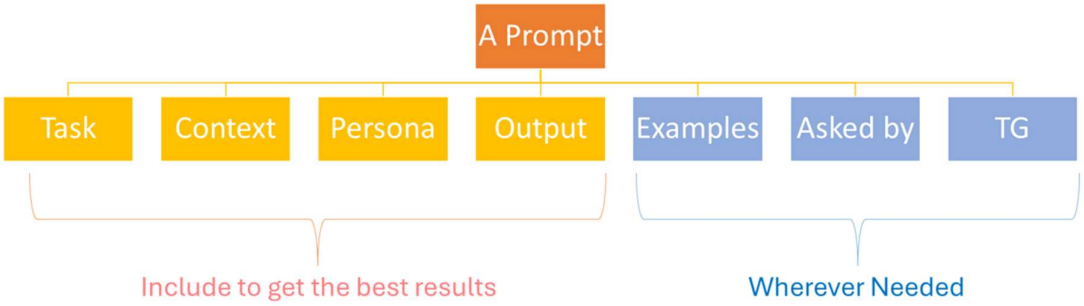
3) Use Azure Open AI APIs to build your application

Submission

Put your application code (.py file) in the submission folder "Prompt Engineering"

General Tips to help with the assignments:

1. Use the prompt engineering best practices learnt in class, summarized below:



2. **Review Prompt Types**
   https://www.promptingguide.ai/
3. **Start with a Simple Prompt**
   - Begin with a single prompt that tries to gather all data and produce output.
   - They'll quickly see if the output is missing details or is disorganized.
4. **Iterative Prompting**
   - Encourage them to **iterate** on the prompt:
     - Add instructions to check for missing info.
     - Add formatting/style guidelines.
     - Add domain constraints (e.g., valid insurance policy numbers, feasible income ranges).

5. **Error Handling**
    - Make sure the prompt politely **asks for corrections** if a user supplies contradictory data (e.g., a date of birth that doesn't make sense).
6. **Be Creative**
    - Customize the final output style (letter, form, summary, bullet points) by specifying the desired format in the prompt.
    - Experiment with formality levels, professional jargon, or layman's terms.
7. **Time Management**
    - Each assignment can fit in a one-hour window if students:
        - Spend ~20 minutes crafting the initial prompt.
        - Spend ~20 minutes refining and testing with sample data.
        - Spend ~20 minutes finalizing the structured output and discussing improvements.

2.1. Hospital Admission Letter Generator
Assigned to: Abhimanyu, Amol, Ansh Tyagi, Bhavesh Patil

Overview

Build a prompt-based "application" to generate a **Hospital Admission Letter** from **unstructured patient input**. The goal is to ensure the prompt captures all necessary details (patient name, date of birth, diagnosis, doctor's name, admission date, and reason for admission).

Steps

1. **Initial Prompt Design**
    - Start with a basic prompt (e.g., "Generate a hospital admission letter from the following information…").
    - Let the user provide some incomplete input (e.g., just the patient name and reason for admission).
2. **Identify Missing Information**
    - The prompt (or chain of prompts) should detect missing fields (like date of birth, doctor's name, etc.) and explicitly **ask** the user to supply them (e.g., "Please provide the doctor's name.").
3. **Refined Prompt with Conditional Logic**
    - Refine the prompt so that it repeatedly checks for completeness. If any field is still missing, it prompts the user again.

4. **Generate the Final Admission Letter**
    - Once all required information is provided, the prompt generates a structured, professional admission letter.

Learning Objectives

- Understand how to prompt for **missing fields** and handle incomplete data.
- Practice refining prompts to produce a **polished** document.
- Experience the back-and-forth nature of prompt-driven interactions.

2.2. Medical Prescription Summary

Assigned to: Breema, Darshit, Dipti Vaidya, Gajanan

Overview

This "application" will generate a **Medical Prescription Summary** based on partial patient and medication data. It demonstrates how to handle domain-specific terminology and ensure completeness (e.g., dosage, frequency).

Steps

1. **Set Up a Conversation Flow**
    - The first prompt instructs the model to **gather patient details** (name, age) and medication details (drug name, dosage, frequency, duration).
2. **Iterative Query**
    - If any piece of information is missing or unclear, the prompt automatically asks the user for clarification.
    - Example: If the user forgets to mention dosage, the model asks, "Please provide the prescribed dosage (e.g., 500 mg)."
3. **Output Format**
    - The final output is a **Prescription Summary** that clearly lists:
        - Patient name
        - Age and relevant medical history (if any)
        - Medication details (drug name, dosage, frequency)
        - Special instructions
4. **Role-Play Variation**
    - Practice different prompts simulating pharmacists or nurses needing a structured summary for patient records.

Learning Objectives

- Use **iterative prompting** to collect domain-specific data (medication dosage, schedule).
- Practice structuring the output in a **clear, professional** format.
- Experience creating domain-aware prompts for healthcare use cases.

2.3. Insurance Claim Form Assistant (Healthcare-Finance Intersection)

Assigned to: Jeevan, Jinav Gala, Jugal, Mayur

Overview

This assignment blends **healthcare** and **finance** by creating an "application" that generates an **Insurance Claim Form**. The "application" will ask for policy details, claimant details, and treatment information, then produce a well-formatted claim form.

Steps

1. **Check Required Fields**
    - Policy number, claimant's name, date of treatment, reason for claim, amount claimed, etc.
2. **Craft a Prompt That Validates Inputs**
    - In the prompt, specify: "If any field is missing or incomplete, ask the user to provide it."
    - Example user input: "Policy #12345, my name is John Doe, and I had a knee surgery."
    - The model should respond: "What is the date of your surgery? Please provide the total cost of treatment. Also provide your date of birth."
3. **Generate the Structured Claim Form**
    - Once all fields are collected, the prompt instructs the model to generate a final claim document.
    - The output should look like a standard insurance form with labeled sections.
4. **Formatting Requirements**
    - Refine the prompt to **format** the output in a tabular or bullet-based layout suitable for submission to an insurance company.

Learning Objectives

- Combine **health and finance data** requirements in one prompt application.
- Implement **error handling** for incomplete or inconsistent fields.

- Learn to output complex multi-section documents through iterative prompt refinement.

2.4. Bank Loan Application Assistant
Assigned to: Nidhi, Pratik Lokhande, Rahul, Ruban Perumal

Overview

Create a prompt-based "assistant" that guides a user through a **Bank Loan Application**. This covers personal and financial details required for the application, ensuring completeness before generating the final loan request form.

Steps

1. **Initial Prompt**
    - The model greets the user and states it will help fill out a loan application.
    - "I need some details. Please provide your name, address, and annual income."
2. **Collect Additional Data**
    - The model recognizes missing fields (loan amount requested, duration, credit score, etc.) and prompts for them.
3. **Refining for Professional Tone**
    - Add instructions to generate a polite, professional final letter.
    - Example: "Use a formal but friendly tone. Include a short introduction, the main request details, and a concluding paragraph thanking the bank."
4. **Final Loan Application Output**
    - The prompt merges collected details into a single structured form/letter.
    - You may add "If any part is unclear, ask me for more info before generating the letter" to ensure completeness.

Learning Objectives

- Learn how to build a **step-by-step data collection** conversation for finance applications.
- Control the **tone and style** of the output via prompt instructions.
- Experience how prompt engineering can eliminate coding for **basic form creation**.

2.5. Personal Financial Plan Generator
Assigned to: Santosh Shinde, Shubhangi, Shruti, Sonu

Overview

This “application” helps a user generate a **personal financial plan** by walking them through their income, expenses, financial goals, and risk tolerance. It emphasizes the use of **conditional logic** based on user inputs (e.g., if they have children, include education savings).

Steps

1. **Discovery Prompt**
    - The model starts by asking: “What is your monthly income? What are your monthly expenses? Do you have any specific financial goals (like retirement, buying a house, or saving for college)?”
2. **Conditional Paths**
    - If the user says they have children, the model follows up with, “Would you like to include a college savings plan?”
    - If the user has no children, it omits that section.
3. **Handling Missing or Conflicting Information**
    - The model carefully checks if the user’s stated expenses exceed income and might prompt further clarifications: “You mentioned your expenses are higher than your income. Do you have any additional sources of funds or debt details to consider?”
4. **Final Financial Plan**
    - The plan includes a breakdown of suggested savings, investments, and advice for future planning.
    - Refine prompts to **summarize** in bullet points or a structured outline.

Learning Objectives

- Incorporate **conditional logic** based on user responses.
- Showcase how to generate **customized outputs** for different user scenarios.
- Practice prompting for complex data to create a concise, actionable plan.
